

Open the black box — from any model

Deck 1 read the model directly (coefficients, tree splits). But a random forest, a boosting model, a neural net — you *can't* read those.

Model-agnostic tools work on *any* fitted model and answer two questions:

- ▶ **How much** does a feature matter? → **Permutation Feature Importance (PFI)**.
- ▶ **How** does a feature act on the prediction? → **feature effects (PDP / ICE)**.

Running example continues: a **bike-sharing** random forest (deck 1's data).

Outline

Permutation feature importance

Feature effects: PDP & ICE

How much does a feature matter?

Make a feature useless and watch the error rise.

Permutation feature importance: the idea

Make feature X_j useless by permuting its column (shuffle its values across rows): same marginal distribution, but its link to the target is broken. Then see how much the error rises:

$$\text{PFI}_j = \underbrace{\mathbb{E}[L(\hat{f}(X_j \text{ permuted}), Y)]}_{\text{error after}} - \underbrace{\mathbb{E}[L(\hat{f}(X), Y)]}_{\text{baseline error}}.$$

- ▶ Big rise $\Rightarrow$ the model leaned on X_j . Near zero $\Rightarrow$ it didn't.
- ▶ **Permute, don't drop** — the model is already trained; we can't remove a feature without refitting a *different* model.
- ▶ Repeat the shuffle several times and average (it's random).

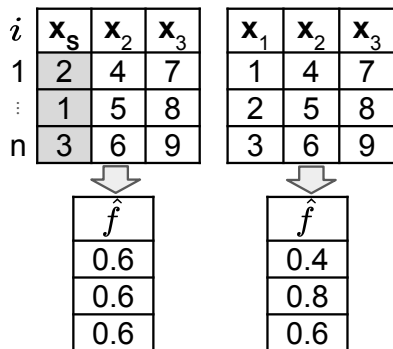
Permutation importance, step by step

i	$\mathbf{x}_s$	$\mathbf{x}_2$	$\mathbf{x}_3$	$\mathbf{x}_1$	$\mathbf{x}_2$	$\mathbf{x}_3$
1	2	4	7	1	4	7
$\vdots$	1	5	8	2	5	8
n	3	6	9	3	6	9

Perturb (permute the feature) $\rightarrow$ **predict** on both versions $\rightarrow$ **aggregate** the change in loss, averaged over rows and repeats.

Source: LMU IML (CC BY 4.0).

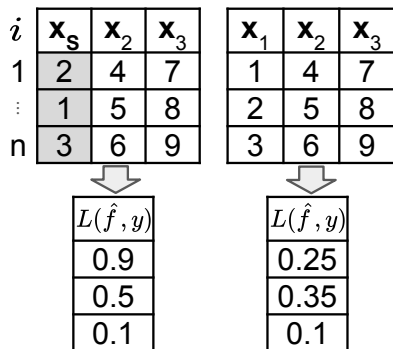
Permutation importance, step by step



Perturb (permute the feature) $\rightarrow$ **predict** on both versions $\rightarrow$ **aggregate** the change in loss, averaged over rows and repeats.

Source: LMU IML (CC BY 4.0).

Permutation importance, step by step



Perturb (permute the feature) → **predict** on both versions → **aggregate** the change in loss, averaged over rows and repeats.

Source: LMU IML (CC BY 4.0).

Permutation importance, step by step

i	$\mathbf{x}_s$	$\mathbf{x}_2$	$\mathbf{x}_3$	$\mathbf{x}_1$	$\mathbf{x}_2$	$\mathbf{x}_3$	ΔL
1	2	4	7	1	4	7	0.65
$\vdots$	1	5	8	2	5	8	0.15
n	3	6	9	3	6	9	0

$L(\hat{f}, y)$		$L(\hat{f}, y)$
0.9		0.25
0.5	-	0.35
0.1		0.1

Perturb (permute the feature) → **predict** on both versions → **aggregate** the change in loss, averaged over rows and repeats.

Source: LMU IML (CC BY 4.0).

Permutation importance, step by step

i	$\mathbf{x}_s$	$\mathbf{x}_2$	$\mathbf{x}_3$	$\mathbf{x}_1$	$\mathbf{x}_2$	$\mathbf{x}_3$	ΔL
1	2	4	7	1	4	7	0.65
$\vdots$	1	5	8	2	5	8	0.15
n	3	6	9	3	6	9	0

= 0.267

Perturb (permute the feature) $\rightarrow$ **predict** on both versions $\rightarrow$ **aggregate** the change in loss, averaged over rows and repeats.

Source: LMU IML (CC BY 4.0).

Permutation importance, step by step

1	i	$\mathbf{x}_s$	$\mathbf{x}_2$	$\mathbf{x}_3$	$\mathbf{x}_1$	$\mathbf{x}_2$	$\mathbf{x}_3$	ΔL	
	1	2	4	7	1	4	7	0.65	
	$\vdots$	1	5	8	2	5	8	0.15	$= 0.267$
	n	3	6	9	3	6	9	0	
m	i	$\mathbf{x}_s$	$\mathbf{x}_2$	$\mathbf{x}_3$	$\mathbf{x}_1$	$\mathbf{x}_2$	$\mathbf{x}_3$	ΔL	
	1	3	4	7	1	4	7	0.85	
	$\vdots$	2	5	8	2	5	8	0	$= 0.4$
	n	1	6	9	3	6	9	0.35	

$\widehat{PFI}_s = \frac{1}{2} (0.267 + 0.4)$

Perturb (permute the feature) → **predict** on both versions → **aggregate** the change in loss, averaged over rows and repeats.

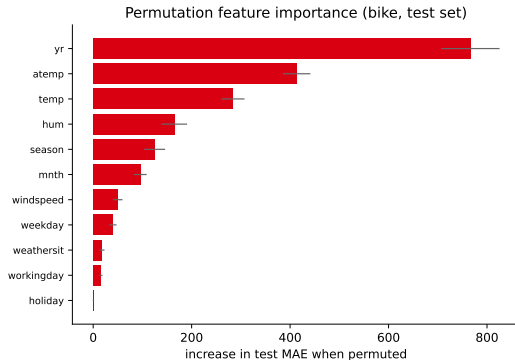
Source: LMU IML (CC BY 4.0).

PFI on the bike model

Permuting each feature on the **test set**, measured as the rise in MAE:

- ▶ `yr`, `atemp`/`temp` dominate. By hand: baseline MAE ≈ 455 ; shuffle `yr` $\rightarrow$ MAE ≈ 1225 ; $\text{PFI}(\text{yr}) \approx \mathbf{770}$.
- ▶ Calendar/weather minutiae barely matter.
- ▶ Error bars = variation over repeated shuffles.

Model-agnostic: same recipe would work on boosting, an SVM, a neural net.



Train or test? They answer different questions

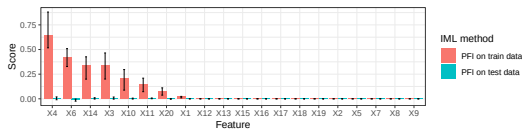
Predict-first: an xgboost model overfits noise. Which features does PFI flag — on train vs. on test?

Train or test? They answer different questions

Predict-first: an xgboost model overfits noise. Which features does PFI flag — on train vs. on test?

- ▶ **Train PFI** highlights whatever the model *overfit* to (spurious links).
- ▶ **Test PFI** shows what actually *generalizes* — here, nothing.

For “which features help the model generalize?”, compute PFI on **test** data.



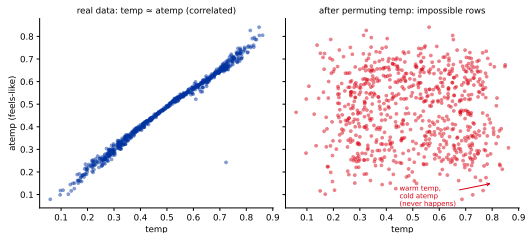
Source: LMU IML (CC BY 4.0).

The correlation trap

Permuting a feature that is **correlated** with others creates **impossible rows** (e.g. a hot temp with a freezing atemp).

- ▶ The model is then scored on inputs it *never* sees in reality (extrapolation).
- ▶ $\Rightarrow$ PFI can **inflate** the importance of correlated features.

Note the flip: correlation *deflated* impurity importance (deck 1); here it *inflates* PFI. **One fix:** conditional variants (CFI) permute a feature *within groups of similar rows*, so the joint distribution stays realistic.



Bike: temp/atemp corr. 0.99; permuting temp makes impossible rows.

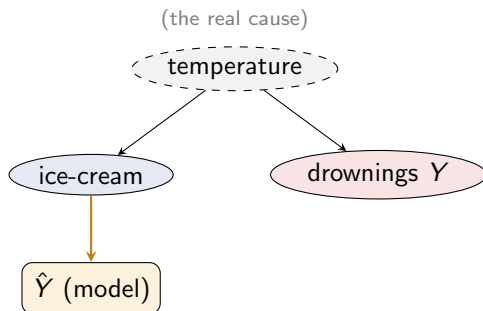
Important $\neq$ causal

Summer heat drives both **ice-cream sales** and **drownings**. A model predicting drownings from ice-cream sales finds ice cream **highly important**.

- ▶ PFI is right: the *model* relies on ice cream.
- ▶ But **banning ice cream won't stop drownings** — it isn't the cause.

Importance tells you what the **model uses**, not what *causes* the outcome.

In our data: yr is the #1 feature, yet the year doesn't *cause* rides — it proxies the service growing. (*Confounding — a different problem from the trap's impossible inputs.*)



Model uses the *proxy* (ice cream), not the cause (temperature).

From “how much” to “how”

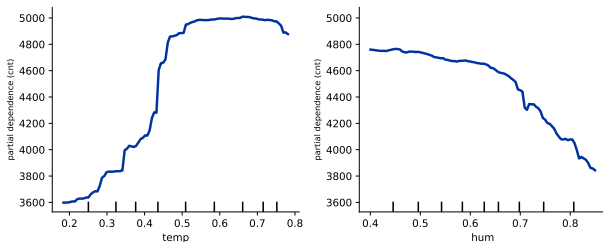
Importance is one number; effects show the *shape* of a feature's influence.

Partial dependence (PDP): the average effect

Vary one feature across its range, average the model's prediction over all other rows, and plot the result — the **average effect** of that feature.

- ▶ temp: rentals climb with warmth, then plateau.
- ▶ hum: high humidity suppresses rentals.

Partial dependence: how the prediction moves with a feature



ICE: one curve per row — and what PDP hides

ICE draws one curve *per observation*;
the **PDP** is **their average**.

Predict-first: the PDP (orange) is **flat**
— does the feature not matter?

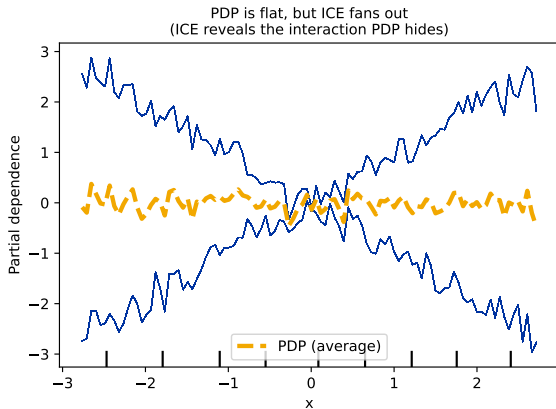
ICE: one curve per row — and what PDP hides

ICE draws one curve *per observation*;
the **PDP** is **their average**.

Predict-first: the PDP (orange) is **flat**
— does the feature not matter?

No. The ICE curves **fan out**: x 's effect
**flips sign depending on another
feature** — rising for half the rows,
falling for the other half, so the
opposing effects **cancel in the
average**. PDP hid that interaction; ICE
exposes it — exactly the kind of
interaction a *linear* model couldn't
represent (deck 1).

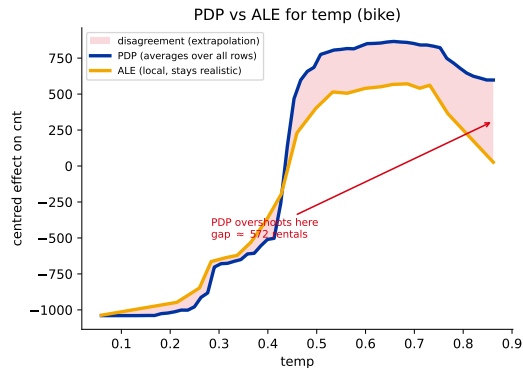
You can even put a number on it: Friedman's **H-statistic** measures how much of a feature's effect
comes from *interactions* rather than acting on its own.



PDP shares PFI's correlation trap — and ALE fixes it

To average over the other features, PDP also evaluates the model on **unrealistic combinations** when features are correlated (same extrapolation as PFI, slide 7).

ALE (Accumulated Local Effects) fixes this: it averages *local* changes within small windows of the feature, staying inside the real data $\Rightarrow$ a correlation-robust effect curve.



Bike temp: PDP overshoots at the extremes; ALE stays realistic.

In sklearn

```
from sklearn.inspection import permutation_importance,
    PartialDependenceDisplay

# PFI on the TEST set -- works for ANY fitted model
r = permutation_importance(model, X_test, y_test, n_repeats=20,
                           scoring="neg_mean_absolute_error")
r.importances_mean          # rise in MAE per feature = importance

# feature effects: PDP (average) + ICE (one line per row)
PartialDependenceDisplay.from_estimator(model, X, ["temp", "hum"])
    # PDP
PartialDependenceDisplay.from_estimator(model, X, ["temp"], kind="
    both")    # PDP + ICE
```

Recap

- ▶ **Model-agnostic** tools open any black box.
- ▶ **PFI** (*how much*): permute a feature, measure the error rise; compute it on **test** data; watch the **correlation trap**; and remember **important** $\neq$ **causal**.
- ▶ **PDP** (*how*): the average effect shape. **ICE**: one curve per row — **PDP is their average**, and ICE reveals interactions the PDP hides.
- ▶ PDP shares PFI's extrapolation issue; **ALE** is the correlation-robust fix.

Next: SHAP — one method that explains a *single* prediction *and* rolls up to global importance & effects, with a rich set of plots.